

Unit-5: Interface, Abstract Class and Exception Handling

Unit-5: Topics and Subtopics Table

Main Topic	Subtopics	What You Learn
1. Interface	Interface concept	Pure abstraction, contract-based design
	Defining interface	Using interface keyword
	Implementing interface	Using implements keyword
	Multiple inheritance using interface	Achieving multiple inheritance in Java
	Partial interface	Partial implementation using abstract class
	Interface vs class	Difference and usage
	Real-world use of interface	Designing flexible systems
2. Abstract Class	Abstract class concept	Partial abstraction
	Abstract methods	Methods without body
	Defining abstract class	Using abstract keyword
	Implementing abstract methods	Child class implementation
	Abstract vs Interface	Key differences
	Real-world use	Template-based design
3. Exception Handling	Exception concept	Runtime error handling
	Types of exceptions	Checked and Unchecked
	try block	Risky code
	catch block	Handle exception
	finally block	Always executes
	throw keyword	Manually throw exception
	throws keyword	Declare exception
	User-defined exception	Custom exception class
	Exception flow	Execution process
	Real-world examples	Robust application design

Unit-5: Foundation (Build Intuition Before Concepts)

This section gives **core understanding + mental model** before starting Interface, Abstract Class, and Exception Handling.

Why This Unit Exists (Big Picture)

In real software:

Code should be:

- ✓ Flexible
- ✓ Reusable
- ✓ Safe (error-free)
- ✓ Easy to extend

Unit-5 solves these problems using:

Concept	Purpose
Interface	Define rules (what to do)
Abstract Class	Provide partial implementation
Exception Handling	Handle runtime errors

🧠 Core Idea (Simple Intuition)

Think in 3 layers:

1. WHAT to do → Interface
2. HOW partially → Abstract Class
3. What if error? → Exception Handling

◆ 1. Interface Intuition (Real Thinking)

🔗 Interface = **Rules / Contract**

Real-World Example

Remote Control

Buttons: Power ON, Volume Up, Channel Change

🔗 Remote says **WHAT actions are available**

🔗 It does NOT define how TV works internally

Programming Thinking

Interface = Only method declaration

No implementation (mostly)

◆ 2. Abstract Class Intuition

🔗 Abstract Class = **Half complete blueprint**

Real-World Example

Vehicle

Common: start(), stop()

Different: fuelType() → Car vs Bike

🔗 Some methods defined

🔗 Some methods left for child class

Programming Thinking

Abstract class = Partial implementation

◆ 3. Exception Handling Intuition

🔗 Exception = **Runtime problem**

Real-World Example

ATM Machine

Errors: Wrong PIN, Insufficient balance

Network failure

🔗 System should NOT crash

🔗 It should handle errors properly

Programming Thinking

Exception Handling = Manage runtime errors safely

🔥 Core Difference (Very Important)

Concept	Focus
Interface	What to do
Abstract Class	How partially
Exception Handling	What if something goes wrong

☐ Mental Model (Important for Understanding)

Design Phase



Interface → Defines rules

Implementation Phase



Abstract Class → Gives partial logic

Execution Phase



Exception Handling → Handles errors

🔍 Why Java Needs These Concepts

Without these:

Problem	Solution
Code duplication	Interface & abstraction
Tight coupling	Interface
Inflexible design	Abstract class
Program crash	Exception handling

⚙️ Real Software Example (Combined)

Online Payment System

Interface → Payment
processPayment()

Abstract Class → OnlinePayment
validateUser()

Child Classes → UPI, Card, NetBanking
implement processPayment()

Exception Handling →
handle invalid payment, network error

📖 Key Terms You Must Know

Term	Meaning
Contract	Rules defined by interface
Abstraction	Hiding implementation
Implementation	Writing actual code
Runtime error	Error during execution
Exception	Object representing error
Robust system	System that handles errors

🧠 One-Line Summary (Memory Trick)

Interface → RULES

Abstract Class → PARTIAL IMPLEMENTATION

Exception Handling → ERROR CONTROL

Unit-5: Interface

Interface is a **core concept of abstraction in Java**. It is used to design systems where **rules are fixed but implementation can vary**.

1. Interface Concept (Pure Abstraction, Contract-Based Design)

Definition: An interface is a completely abstract structure that defines a set of rules (methods) that a class must follow.

☞ It provides a **contract**:

- “If you implement this interface, you must provide these methods”

Core Idea

Interface = Contract (Rules)

Class = Implementation (Actual Work)

Key Characteristics

Feature	Explanation
Abstraction	Only method declarations (no implementation)
Contract-based	Forces class to follow rules
No object creation	Cannot instantiate interface
Methods	Public and abstract by default
Variables	public static final (constants)

Real Understanding

Example: **USB Port**

USB Interface:

```
connect()
transferData()
```

Devices like:

- Keyboard
- Mouse
- Pen drive

☞ All follow same rules but work differently

Code Example

```
interface Animal {
    void sound();
}
```

2. Defining Interface (Using interface keyword)

Syntax

```
interface InterfaceName {
    returnType methodName();
}
```

Example

```
interface Vehicle {
    void start();
    void stop();
}
```

Important Rules

Rule	Explanation
Methods are abstract	No body allowed
Methods are public	Automatically
Variables are constants	final and static

Cannot use constructors

Interface cannot create objects

Example with Variables

```
interface Test {  
    int value = 10; // constant  
}
```

Equivalent to: `public static final int value = 10;`

3. Implementing Interface (Using implements keyword)

Concept: A class uses **implements keyword** to follow the rules defined in an interface.

👉 The class must implement **all methods**

Syntax

```
class ClassName implements InterfaceName {  
    public void methodName() {  
        // implementation  
    }  
}
```

Example

```
interface Animal {  
    void sound();  
}
```

```
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

Important Rule

Method must be:

- ✓ public
- ✓ same name
- ✓ same parameters

Full Example

```
interface Shape {  
    void draw();  
}  
  
class Circle implements Shape {  
    public void draw() {  
        System.out.println("Drawing Circle");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Shape s = new Circle();  
        s.draw();  
    }  
}
```

4. Multiple Inheritance using Interface

Concept

Java does not allow:

class C extends A, B ✗

But allows:

class C implements A, B ✓

Why?

Avoids **ambiguity problem (Diamond Problem)**.

Syntax

```
class ClassName implements Interface1, Interface2 {  
}
```

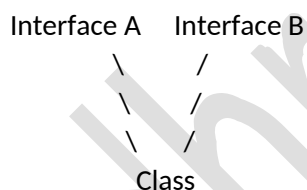
Example

```
interface A {  
    void show();  
}
```

```
interface B {  
    void display();  
}
```

```
class Test implements A, B {  
    public void show() {  
        System.out.println("Show method");  
    }  
    public void display() {  
        System.out.println("Display method");  
    }  
}
```

Flow



5. Partial Interface (Using Abstract Class)

Concept: If a class does **not implement all methods**, it must be declared **abstract**.

Example

```
interface Animal {  
    void eat();  
    void sound();  
}
```

```
abstract class Dog implements Animal {  
    public void eat() {  
        System.out.println("Dog eats");  
    }  
}
```

```
}  
}
```

Here:

sound() → not implemented

Dog → abstract class

Why Needed?

Sometimes:

- You want **partial implementation**
- Full implementation is done in **child class**

Complete Example

```
class Puppy extends Dog {  
    public void sound() {  
        System.out.println("Puppy barks");  
    }  
}
```

6. Interface vs Class

Difference Table

Feature	Interface	Class
Purpose	Define rules	Implement logic
Methods	Abstract	Abstract + concrete
Variables	Constants only	Normal variables
Object creation	Not allowed	Allowed
Inheritance	Multiple	Single

Interface vs Abstract Class

Feature	Interface	Abstract Class
Methods	Abstract	Abstract + concrete
Variables	Only constants	Normal variables
Constructor	Not allowed	Allowed
Multiple inheritance	Yes	No

7. Real-World Use of Interface (Flexible System Design)

Concept: Interfaces are used to **build systems that can easily change and extend**.

Example: Payment System

Interface → Payment

```
pay()
```

Classes:

UPI

CreditCard

NetBanking

Code Example

```
interface Payment {  
    void pay();  
}
```

```
class UPI implements Payment {
```

```

public void pay() {
    System.out.println("UPI Payment");
}
}

```

```

class Card implements Payment {
    public void pay() {
        System.out.println("Card Payment");
    }
}

```

Usage

```

Payment p;
p = new UPI();
p.pay();

```

```

p = new Card();
p.pay();

```

Benefit

Same interface → Different implementations

8. Advanced Understanding (Important)

Interface Reference Concept

```
Payment p = new UPI();
```

Part	Meaning
Payment	Reference type
UPI	Object type

Why This is Powerful

- Code becomes **flexible**
- You can switch implementation easily

9. Design Advantage of Interface

Advantage	Explanation
Loose coupling	Classes are independent
Scalability	Easy to add new classes
Maintainability	Less code change
Standardization	Fixed method structure

10. Summary Understanding

Interface:

- ✓ Defines rules
- ✓ No implementation
- ✓ Enables multiple inheritance
- ✓ Used for flexible design

Unit-5: Abstract Class

Abstract class is used when you want to **define a base structure with partial implementation**.

It sits between **interface (only rules)** and **normal class (full implementation)**.

1. Abstract Class Concept

Definition: An abstract class is a class that cannot be instantiated and may contain both abstract (no body) and concrete (with body) methods.

Core Idea

Abstract Class = Partial Abstraction

Why We Need Abstract Class

Sometimes:

- Some behavior is **common**
- Some behavior is **different**

🔗 Abstract class allows:

- Define common logic
- Leave some methods for child class

Real-World Example

Vehicle System

Vehicle

- |
- | start() → same for all
- | stop() → same for all
- | fuelType() → different

Code Example

```
abstract class Vehicle {  
    void start() {  
        System.out.println("Vehicle starts");  
    }  
    abstract void fuelType();  
}
```

2. Abstract Methods

Definition : An abstract method is a method declared without implementation (no body).

Syntax

```
abstract returnType methodName();
```

Example

```
abstract class Animal {  
    abstract void sound();  
}
```

Important Rules

Rule	Explanation
No body	Only declaration
Must be inside abstract class	Required
Must be implemented in child class	Mandatory

3. Defining Abstract Class

Syntax

```
abstract class ClassName {  
    abstract void method1();  
    void method2() {
```

```
// normal method
}
}
```

Example

```
abstract class Shape {
    abstract void draw();
    void display() {
        System.out.println("Shape display");
    }
}
```

Key Points

Feature	Explanation
Cannot create object	new Shape() ✕
Can have constructors	✓
Can have variables	✓
Can have normal methods	✓

4. Implementing Abstract Methods

Concept: A child class must **implement all abstract methods** of parent class.

Syntax

```
class ChildClass extends AbstractClass {
    void methodName() {
        // implementation
    }
}
```

Example

```
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

Full Program

```
abstract class Animal {
    abstract void sound();
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
class Test {
    public static void main(String[] args) {
        Animal a=new Dog();
        a.sound();
    }
}
```

```
}  
}
```

Important Rule

If class does NOT implement all methods → it must be abstract

5. Abstract Class vs Interface

Detailed Comparison

Feature	Abstract Class	Interface
Abstraction	Partial	Full (conceptually)
Methods	Abstract + concrete	Mostly abstract
Variables	Normal variables allowed	Only constants
Constructor	Allowed	Not allowed
Multiple inheritance	Not supported	Supported
Keyword	abstract	interface

Understanding Difference

Interface → only rules

Abstract Class → rules + partial logic

6. Real-World Use of Abstract Class

Concept

Abstract class is used when:

- Some logic is **common**
- Some logic must be **customized**

Example: Payment System

Abstract Class → Payment

```
validateUser()
```

Abstract Method:

```
processPayment()
```

Code Example

```
abstract class Payment {  
    void validateUser() {  
        System.out.println("User validated");  
    }  
    abstract void processPayment();  
}  
  
class UPI extends Payment {  
    void processPayment() {  
        System.out.println("UPI Payment");  
    }  
}  
  
class Card extends Payment {  
    void processPayment() {  
        System.out.println("Card Payment");  
    }  
}
```

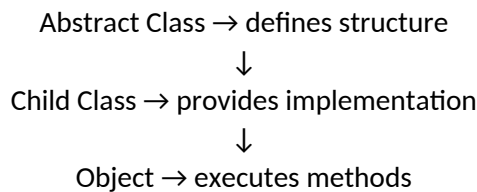
Usage

```

class Test {
    public static void main(String[] args) {
        Payment p=new UPI();
        p.validateUser();
        p.processPayment();
    }
}

```

7. Execution Flow



8. Design Advantages

Advantage	Explanation
Code reuse	Common methods defined once
Flexibility	Child classes customize behavior
Maintainability	Easy to update
Better design	Structured hierarchy

9. Key Understanding

Abstract class:

- ✓ Cannot create object
- ✓ Can have both methods
- ✓ Used for partial implementation
- ✓ Extended using 'extends'

10. Interface vs Abstract Class (Final Clarity)

Interface → 100% rules

Abstract Class → 50% rules + 50% logic

Class → 100% implementation

Unit-5: Exception Theory (Before Exception Handling)

This section builds **strong theoretical understanding** of errors and exceptions.

It helps you clearly understand **why exception handling is needed**.

1. What is Error?

Definition: An **error** is a problem in a program that prevents it from executing properly.

Errors can occur:

- before execution (compile-time)
- during execution (runtime)

Simple Understanding

Error = Something wrong in program

Example

```
int a = 10
```

```
System.out.println(a);
```

Missing ; → Syntax error

2. Characteristics of Errors

Characteristic	Explanation
Occur due to mistakes	Coding or logical mistake
Can stop execution	Program may crash
Some are detectable early	Compile-time errors
Some occur during execution	Runtime errors

3. Types of Errors

1. Compile-Time Errors

Definition: Errors detected by the compiler **before** program runs.

Examples

```
int a = "hello"; // type mismatch
System.out.println("Hello") // missing semicolon
```

Features

Feature	Explanation
Detected early	During compilation
Easy to fix	Compiler shows error
Prevent execution	Program will not run

2. Runtime Errors

Definition: Errors that occur **during** execution of program.

Examples

```
int a = 10/0; // ArithmeticException
int arr[] = new int[3];
arr[5] = 10; // ArrayIndexOutOfBoundsException
```

Features

Feature	Explanation
Occur during execution	After compilation
Harder to detect	Depends on input
Cause program crash	If not handled

3. Logical Errors

Definition: Errors where program runs but gives **wrong** output.

Example: `int sum = a - b;` // should be +

Features

Feature	Explanation
No error message	Runs normally
Wrong result	Incorrect logic
Hardest to detect	Requires testing

4. What is Exception?

Definition: An exception is an abnormal condition that occurs during runtime and disrupts normal flow of program.

Key Idea: Exception = Runtime error object

Example

```
int a = 10/0;
```

This creates: `ArithmeticException`

Difference Between Error and Exception

Feature	Error	Exception
Type	Serious problem	Manageable problem
Handling	Cannot handle easily	Can be handled
Occurrence	System-level	Program-level

5. Characteristics of Exception

Characteristic	Explanation
Occurs at runtime	During execution
Interrupts flow	Stops normal execution
Can be handled	Using try-catch
Represented as object	Belongs to class

6. Types of Exceptions in Java

Java exceptions are divided into **two main categories**:

1. Checked Exceptions

Definition: Exceptions that are checked **at compile time**.

Examples

`IOException`

`SQLException`

`FileNotFoundException`

Features

Feature	Explanation
Checked at compile time	Compiler forces handling
Must handle	try-catch or throws
External conditions	File, database

2. Unchecked Exceptions

Definition: Exceptions that occur **at runtime** and are not checked at compile time.

Examples

`ArithmeticException`

`NullPointerException`

`ArrayIndexOutOfBoundsException`

Features

Feature	Explanation
Occur at runtime	Not checked by compiler
Optional handling	Not mandatory
Programming errors	Logic mistakes

3. Errors (System Errors)

These are also part of Java hierarchy but **not handled normally**.

Examples

`OutOfMemoryError`

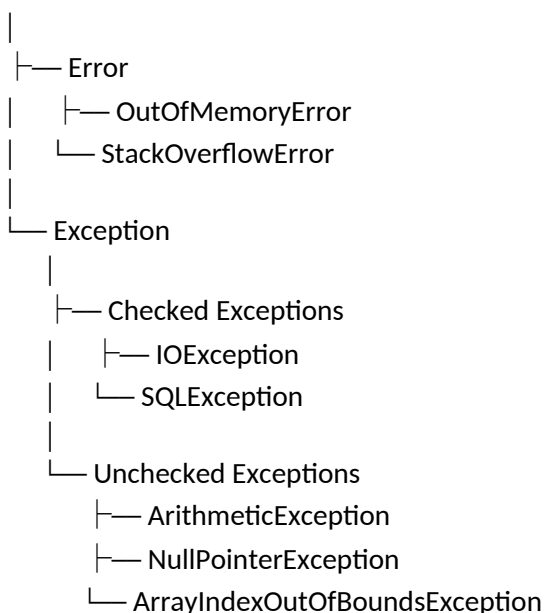
StackOverflowError

Features

Feature	Explanation
Serious problems	System failure
Cannot handle easily	JVM related

7. Exception Hierarchy (Very Important)

Throwable



8. Exception Classes in Java

All exceptions are **classes** derived from:
`java.lang.Throwable`

Common Exception Classes

Exception Class	Description
ArithmeticException	Division by zero
NullPointerException	Using null object
ArrayIndexOutOfBoundsException	Invalid array index
NumberFormatException	Invalid number conversion
IOException	Input/output failure
FileNotFoundException	File not found
ClassNotFoundException	Class missing
IllegalArgumentException	Invalid argument

Example

```
int a = Integer.parseInt("abc");
```

Throws: `NumberFormatException`

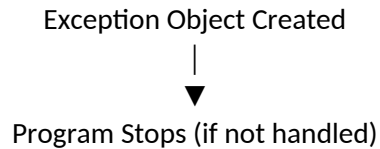
9. Execution Flow of Exception

Normal Execution



Error Occurs





10. Why Exception Handling is Needed

Without handling: Program crashes ✗

With handling: Program continues ✓

11. Final Understanding

Error → Problem in program

Exception → Runtime problem (can handle)

Checked → Compile-time check

Unchecked → Runtime issue

Throwable → Root class

Unit-5: Exception Handling

Exception handling is used to **handle runtime errors safely so that the program does not crash** and continues execution in a controlled way.

1. What is Exception Handling?

Definition: Exception handling is a mechanism in Java to handle runtime errors using special blocks like try, catch, and finally.

Core Idea

Normal Flow → Error Occurs → Handle Error → Continue Program

Without Exception Handling

```
int a=10/0;
```

```
System.out.println("Next line");
```

Output: Exception in thread "main" java.lang.ArithmeticException

☞ Program stops ✗

With Exception Handling

```
try {
```

```
    int a=10/0;
```

```
} catch(ArithmeticException e) {
```

```
    System.out.println("Error handled");
```

```
}
```

```
System.out.println("Next line");
```

Output:

Error handled

Next line

☞ Program continues ✓

2. Types of Exceptions

Java exceptions are mainly divided into **Checked and Unchecked**.

1. Checked Exceptions

These are **checked at compile time**.

☞ Compiler forces you to handle them.

Example

```
import java.io.*;
class Test {
    public static void main(String[] args) throws IOException {
        FileReader f=new FileReader("file.txt");
    }
}
```

Key Understanding

- Occur due to external conditions (file, database)
- Must be handled using try-catch or throws

2. Unchecked Exceptions

These occur **at runtime**.

🔒 Not checked by compiler.

Example

```
int a=10/0;
```

Common Examples

ArithmeticException

NullPointerException

ArrayIndexOutOfBoundsException

Difference

Feature	Checked	Unchecked
Time	Compile-time	Runtime
Handling	Mandatory	Optional
Cause	External	Programming error

3. try Block

Concept: The try block contains code that may cause an exception.

Syntax

```
try {
    // risky code
}
```

Example

```
try {
    int a=10/0;
}
```

Important Points

- Must be followed by **catch or finally**
- Cannot exist alone
- Only one try block allowed per structure

Understanding

try = "This code may cause error"

4. catch Block

Concept: The catch block handles the exception thrown by try block.

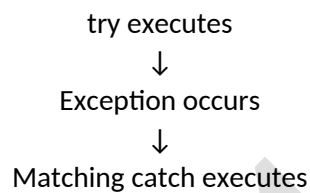
Syntax

```
catch(ExceptionType e) {  
    // handling code  
}
```

Example

```
try {  
    int a=10/0;  
} catch(ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

How It Works



Multiple catch Blocks

```
try {  
    int arr[]=new int[3];  
    arr[5]=10;  
} catch(ArithmeticException e) {  
    System.out.println("Arithmetic error");  
} catch(ArrayIndexOutOfBoundsException e) {  
    System.out.println("Array index error");  
}
```

Important Rules

Rule	Explanation
Order matters	Specific → general
Only one catch executes	Matching one
Multiple catch allowed	✓

5. finally Block

Concept: The **finally block** always executes whether exception occurs or not.

Syntax

```
finally {  
    // always executes  
}
```

Example

```
try {  
    int a=10/2;  
} catch(Exception e) {  
    System.out.println("Error");  
} finally {  
    System.out.println("Always runs");  
}
```

Output:
Always runs

Example with Exception

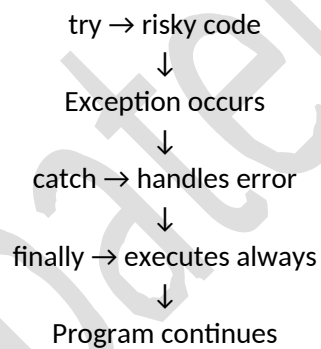
```
try {  
    int a=10/0;  
} catch(Exception e) {  
    System.out.println("Error handled");  
} finally {  
    System.out.println("Always runs");  
}
```

Output:
Error handled
Always runs

Why finally is Used

Purpose	Explanation
Resource cleanup	Close file, database
Guaranteed execution	Always runs
Maintain consistency	Safe program state

6. Complete Flow of Exception Handling



7. Real-World Examples (In-Depth)

Example 1: ATM System

Problem: User enters wrong PIN or insufficient balance

Code

```
class ATM {  
    public static void main(String[] args) {  
        int balance=5000;  
        int withdraw=7000;  
        try {  
            if(withdraw>balance) {  
                throw new ArithmeticException("Insufficient Balance");  
            }  
            balance-=withdraw;  
        } catch(ArithmeticException e) {  
            System.out.println(e.getMessage());  
        } finally {  
            System.out.println("Transaction complete");  
        }  
    }  
}
```

```
}  
}
```

Output

Insufficient Balance
Transaction complete

Understanding

- try → checks condition
- catch → handles error
- finally → always runs

Example 2: Array Access System

Problem: User accesses invalid index

Code

```
class Test {  
    public static void main(String[] args) {  
        int arr[]=new int[3];  
        try {  
            arr[5]=10;  
        } catch(ArrayIndexOutOfBoundsException e) {  
            System.out.println("Invalid index");  
        } finally {  
            System.out.println("Program continues");  
        }  
    }  
}
```

Output

Invalid index
Program continues

Example 3: Division Calculator

```
class Test {  
    public static void main(String[] args) {  
        int a=10,b=0;  
        try {  
            int c=a/b;  
        } catch(ArithmeticException e) {  
            System.out.println("Cannot divide by zero");  
        } finally {  
            System.out.println("End of program");  
        }  
    }  
}
```

Output

Cannot divide by zero
End of program

Example 4: Multiple Exception Handling

```
class Test {
```

```

public static void main(String[] args) {
    try {
        String s=null;
        System.out.println(s.length());
    } catch(NullPointerException e) {
        System.out.println("Null error");
    } catch(Exception e) {
        System.out.println("General error");
    }
}
}

```

Output

Null error

8. Important Behavior Understanding

Case 1: No Exception

```

try {
    System.out.println("No error");
} catch(Exception e) {
    System.out.println("Error");
} finally {
    System.out.println("Finally block");
}

```

Output:

No error

Finally block

Case 2: Exception Occurs

try → error

catch → executes

finally → executes

Case 3: No catch, only finally

```

try {
    int a=10/0;
} finally {
    System.out.println("Finally");
}

```

☞ Exception still occurs, but finally runs

9. Why Exception Handling is Important

Problem	Solution
Program crash	Prevent crash
Bad user experience	Graceful handling
Data loss	Safe execution
Debugging difficulty	Clear error messages

10. Final Understanding

try → risky code

catch → handle error

finally → always executes

Checked → must handle

Unchecked → optional handling

Goal → Program should NOT crash

Unit-5: Advanced Exception Handling Concepts

(throw, throws, User-defined Exception, Exception Flow)

These concepts explain **how exceptions are created, declared, customized, and how they move inside a program.**

1. throw Keyword

Concept: The throw keyword is used to manually create and throw an exception.

🔗 Used when you want to **force an exception based on a condition**

Syntax: throw new ExceptionType("message");

Simple Example

```
class Test {  
    public static void main(String[] args) {  
        int age=16;  
        if(age<18) {  
            throw new ArithmeticException("Not eligible to vote");  
        }  
        System.out.println("Eligible");  
    }  
}
```

Output

Exception in thread "main" java.lang.ArithmeticException: Not eligible to vote

Understanding

Condition → throw → Exception object created → Program stops (if not handled)

Important Points

Point	Explanation
Used inside method/block	✓
Creates exception object	✓
Works with custom logic	✓
Can throw only one exception at a time	✓

2. throws Keyword

Concept: The **throws** keyword is used to declare exceptions that a method might throw.

🔗 It tells the compiler:

"This method may cause exception, handle it outside"

Syntax

```
returnType methodName() throws ExceptionType {  
}
```

Example

```
import java.io.*;  
class Test {  
    static void readFile() throws IOException {  
        FileReader f=new FileReader("file.txt");
```

```

}
public static void main(String[] args) {
    try {
        readFile();
    } catch(IOException e) {
        System.out.println("File error");
    }
}
}
}

```

Understanding

Method → declares exception (throws)

Caller → handles exception (try-catch)

Key Difference (throw vs throws)

Feature	throw	throws
Purpose	Create exception	Declare exception
Used in	Method body	Method signature
Number	One exception	Multiple exceptions
Keyword type	Action	Declaration

3. User-Defined Exception

Concept: A user-defined exception is a custom exception created by the programmer.

🔧 Used when built-in exceptions are not enough

Why Needed

Real-world problems need **custom logic**

Example:

Bank → Insufficient balance

Exam → Marks below passing

Age → Not eligible

Steps to Create User-Defined Exception

1. Create class extending Exception
2. Define constructor
3. Use throw to trigger it

Example

```

class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg);
    }
}

class Test {
    public static void main(String[] args) {
        int age=16;
        try {
            if(age<18) {
                throw new InvalidAgeException("Not eligible");
            }
        } catch(InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

```
}  
}  
}
```

Output

Not eligible

Key Understanding

Custom condition → throw custom exception → handle using catch

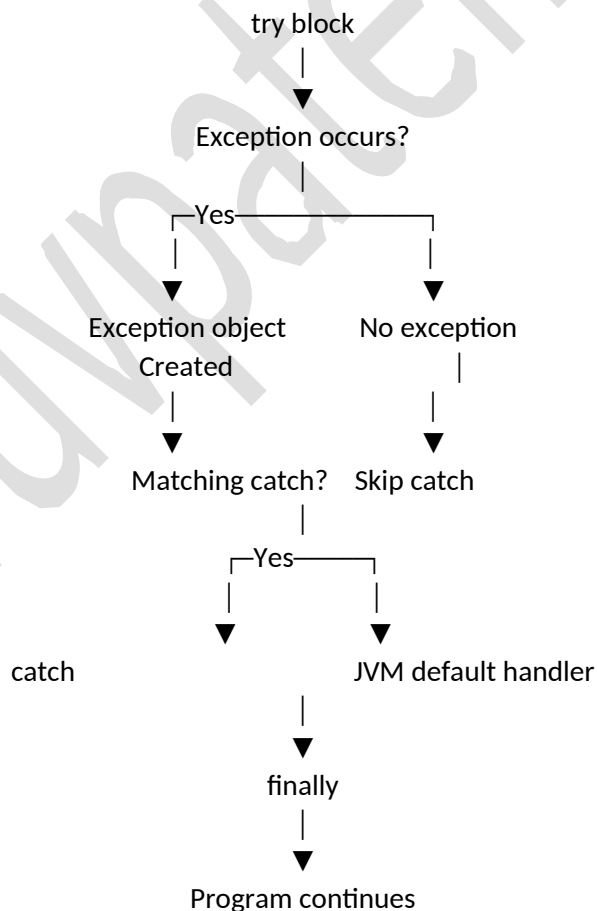
4. Exception Flow (Very Important Concept)

Concept: Exception flow describes **how exception moves inside a program.**

Step-by-Step Flow

1. Program starts
2. try block executes
3. Exception occurs
4. Exception object created
5. JVM searches for matching catch
6. If found → catch executes
7. finally executes
8. Program continues

Flow Diagram



Example Showing Full Flow

```
class Test {  
    public static void main(String[] args) {  
        try {
```



```

        int a=10/0;
    } catch(ArithmeticException e) {
        System.out.println("Handled");
    } finally {
        System.out.println("Finally block");
    }
    System.out.println("Program continues");
}
}

```

Output

Handled
Finally block
Program continues

Case Analysis

Case 1: Exception Handled

try → error → catch → finally → continue

Case 2: Exception Not Handled

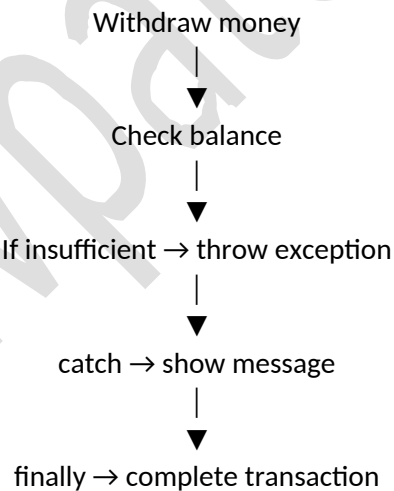
try → error → no catch → JVM default handler → crash

Case 3: No Exception

try → no error → skip catch → finally → continue

Real-World Example (Deep Understanding)

Banking System



Code Example

```

class Bank {
    public static void main(String[] args) {
        int balance=5000;
        int withdraw=7000;
        try {
            if(withdraw>balance) {
                throw new ArithmeticException("Insufficient Balance");
            }
        } catch(ArithmeticException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

```
    } finally {  
        System.out.println("Transaction done");  
    }  
}  
}
```

Output

Insufficient Balance

Transaction done

Final Understanding

throw → creates exception

throws → declares exception

User-defined → custom exception

Flow → how exception moves

Goal:

- ✓ Control errors
- ✓ Prevent crash
- ✓ Maintain program flow